Ödev: Sıfırdan Pokemon CRUD Uygulaması

Amaç

Vite ile sıfırdan bir React + TypeScript projesi oluşturup; **Axios**, **React Query**, **JSON Server**, **Tailwind CSS** ve **DaisyUI** kurarak tam işlevsel bir CRUD (Create, Read, Update, Delete) uygulaması geliştirmek.

---

1. Projeyi Oluştur

```
npm create vite@latest pokemon-crud -- --template react-ts
cd pokemon-crud
npm install
```

Projenin çalıştığını doğrula:

```
npm run dev
```

---

2. Bağımlılıkları Kur

a) Axios + React Query + JSON Server + concurrently

```
npm install axios @tanstack/react-query
npm install -D json-server concurrently
```

b) Tailwind CSS v4 + DaisyUI

```
npm install tailwindcss @tailwindcss/vite daisyui
```

Kurulum adımları:

- `vite.config.ts` içine Tailwind plugin'ini ekle:

```
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

- `src/index.css` dosyasının içeriğini tamamen sil, şunu yaz:

```
@import "tailwindcss";
@plugin "daisyui";
```

c) (Bonus) React Query Devtools

```
npm install -D @tanstack/react-query-devtools
```

3. JSON Server Hazırla

Proje kökünde `db.json` dosyası oluştur. En az 5 başlangıç kaydı ekle:

```
{
  "pokemons": [
    {
      "id": "1",
      "name": "Pikachu",
      "type": "Electric",
      "level": 25,
      "imageUrl": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/
    }
  ]
}
```

`package.json` içindeki `scripts` bölümünü güncelle:

```
{
  "scripts": {
    "dev": "concurrently -c \"blue,green\" -n \"WEB,API\" \"vite\" \"json-server db.json\""
    "api": "json-server db.json",
    "build": "tsc -b && vite build",
    "lint": "eslint ."
  }
}
```

4. Proje Yapısını Oluştur

Aşağıdaki klasör/dosya yapısını kendin kur:

```
src/
├── api/
│   └── axios.ts          # axios instance (baseURL: http://localhost:3000)
├── types/
│   └── pokemon.ts        # Pokemon type tanımı
├── hooks/
│   └── usePokemons.ts    # React Query hook'ları
├── components/
│   ├── PokemonList.tsx
│   ├── PokemonForm.tsx
│   ├── EditPokemonModal.tsx
│   └── DeleteConfirmModal.tsx
├── App.tsx
├── main.tsx
└── index.css
```

---

5. Type Tanımı

`src/types/pokemon.ts`:

```
export type Pokemon = {
  id: string;
  name: string;
  type: string;
  level: number;
  imageUrl?: string;
};
```

---

6. React Query Kurulumu

`src/main.tsx` içinde `QueryClientProvider` ile `App`'i sarmala. `QueryClient` instance'ı yarat.

---

7. CRUD Hook'larını Yaz

`src/hooks/usePokemons.ts` dosyasında şu 5 hook'u tanımla:

Hook	HTTP	Endpoint	Açıklama
`usePokemons()`	GET	`/pokemons`	Tüm listeyi getir
`usePokemon(id)`	GET	`/pokemons/:id`	Tek pokemon detayı
`useCreatePokemon()`	POST	`/pokemons`	Yeni ekle
`useUpdatePokemon()`	PUT veya PATCH	`/pokemons/:id`	Güncelle
`useDeletePokemon()`	DELETE	`/pokemons/:id`	Sil

Dikkat edilecekler:

- Query'lerde `staleTime` ayarı ekle (3-5 dakika).
- Her mutation'ın `onSuccess` callback'inde `queryClient.invalidateQueries` ile cache'i yenile.
- TypeScript tiplerini eksiksiz ver (`Omit<Pokemon, "id">` , `Partial<Pokemon>` vb.).
- `useQuery` 'de `enabled` option'ı ile koşullu fetch'i öğren (id varsa fetch et).

---

8. UI Bileşenleri (DaisyUI ile)

a) Liste Görünümü (`PokemonList.tsx`)

- DaisyUI `card` veya `table` bileşeni.
- Her kartta **Düzenle** ve **Sil** butonları.
- `isLoading` → `loading loading-spinner` .
- `isError` → `alert alert-error` .
- Liste boşsa "Henüz pokemon yok" mesajı.

b) Ekleme Formu (`PokemonForm.tsx`)

- `input` , `select` (type için), `number` input (level için).
- `btn btn-primary` ile "Ekle" butonu.
- Validation: boş isim engellensin.
- `isPending` iken buton disabled olsun.
- Başarı durumunda form temizlensin.

c) Düzenleme Modal'ı (`EditPokemonModal.tsx`)

- DaisyUI `modal` bileşeni.
- Düzenle butonuna basılınca mevcut değerlerle form açılsın.
- Kaydet / İptal butonları.

d) Silme Onayı (`DeleteConfirmModal.tsx`)

- DaisyUI `modal` ile onay sor.
- "Evet, sil" / "Vazgeç" butonları.

---

9. Kullanıcı Deneyimi

- [ ] Tüm loading state'lerinde spinner veya skeleton kullan.
- [ ] Mutation sırasında butonlar disabled (`isPending`).
- [ ] Responsive tasarım: mobil ve desktop'ta düzgün.
- [ ] DaisyUI tema değiştirici ekle (`data-theme` attribute).

Bonus (İsteğe Bağlı)

1. **Arama/Filtre**: İsme göre client-side filtreleme.
2. **Sıralama**: Level'a göre artan/azalan sıralama.
3. **Optimistic Update**: Silme işleminde UI anında güncellensin, hata olursa geri alsın.
4. **Pagination**: JSON Server `_page` ve `_limit` query parametreleri.
5. **React Query Devtools**: Geliştirme sırasında cache'i görselleştir.
6. **Detay Sayfası**: React Router ile `/pokemons/:id` route'u.

Çalıştırma

```
npm run dev
```

- Web: http://localhost:5173
- API: http://localhost:3000/pokemons

Teslim Kriterleri

- [ ] Vite ile sıfırdan kurulum yapılmış.
- [ ] Tailwind v4 + DaisyUI çalışıyor.
- [ ] JSON Server `concurrently` ile aynı komutta çalışıyor.
- [ ] 5 CRUD hook'u çalışıyor (list, detail, create, update, delete).
- [ ] Liste, ekleme formu, düzenleme modal'ı, silme onayı mevcut.
- [ ] Loading & error state'leri handle ediliyor.
- [ ] TypeScript hataları yok (`npm run build` temiz geçiyor).
- [ ] ESLint uyarıları yok (`npm run lint` temiz).

Faydalı Linkler

- Vite
- TanStack Query Docs
- Axios
- JSON Server
- Tailwind CSS v4 + Vite
- DaisyUI Components